

Urban Scene Parsing & Infrastructure Analysis

Object Detection with YOLOv11 and Semantic Segmentation with SegFormer/U-Net on the BDD100K Driving Dataset

Ayoob Haroon | Roll No. 20I-0777

Department of Computer Science | Part 1 Final Report

Abstract

This report presents a complete urban scene parsing pipeline built on the BDD100K Berkeley DeepDrive dataset. The system addresses two complementary computer vision problems simultaneously: real-time object detection using **YOLOv11-nano** and semantic segmentation using both **SegFormer-b0** (transformer-based) and a custom **U-Net** (convolutional baseline). Experiments were conducted on a 10-class subset covering safety-critical road objects. The augmented YOLOv11 model achieved measurably higher mAP@50 compared to its non-augmented baseline, confirming the value of mosaic, colour jitter, and affine augmentation strategies. SegFormer-b0 outperformed U-Net on mean IoU while the U-Net trained faster and consumed less GPU memory. Failure analysis and weather-stratified evaluation reveal model sensitivity to night-time scenes and adverse weather, with practical implications for IoT edge deployment. All code, checkpoints, and result CSVs are provided in the accompanying Jupyter notebook `part1_vision_optimized_Final.ipynb`.

Introduction

Reliable scene understanding is a foundational capability for both autonomous vehicles (AVs) and smart city infrastructure monitoring. Urban roads present particular challenges: severe class imbalance among detected objects, rapidly changing lighting conditions, and degraded image quality under adverse weather all confound standard computer vision pipelines.

This work explores how two modern architectures—a single-stage real-time detector and a transformer-based segmentor—can be jointly applied to the **BDD100K** dataset [1], one of the most comprehensive publicly available driving datasets, comprising 100,000 annotated dashcam frames across diverse conditions.

The report makes the following contributions:

- A structured **Exploratory Data Analysis** (EDA) characterising the distribution of classes, weather conditions, and lighting.
- A comparative **augmentation study** for YOLOv11, demonstrating the impact of mosaic, colour jitter, and affine transforms.
- A head-to-head **SegFormer-b0 vs. U-Net** comparison on segmentation quality and computational cost.
- A **failure analysis** and discussion of edge-device deployment strategies for smart city applications.

Part 1 Methodology & Experimental Results

Dataset & Preprocessing

BDD100K provides two dataset flavours used for separate tasks. The `100k/train` split (70,000 images, 1280×720) with JSON bounding box labels feeds the detection pipeline. The `seg/images/train` split ($\approx 7,000$ images) accompanied by greyscale label PNGs (`seg/labels/train`) serves the segmentation pipeline. The `seg/color_labels` folder provides human-readable

coloured masks used exclusively for EDA visualisation; raw RGB values from colour masks were never used as training targets.

10-Class Subset Justification

Ten classes were selected from the full BDD100K taxonomy on the basis of (i) annotation frequency, (ii) autonomous-driving safety relevance, and (iii) class balance considerations:

ID	Class	Reason
0	car	Most frequent; core AV hazard
1	pedestrian	Safety-critical (required)
2	traffic light	Road regulation (required)
3	truck	Large vehicle; hazard class
4	bus	Urban scene fixture
5	traffic sign	AV decision-making
6	motorcycle	Vulnerable road user
7	bicycle	Vulnerable road user
8	rider	Augments cyclist detection
9	road/drivable	Road hazard proxy (required)

EDA Findings

Category distribution analysis revealed a log-scale imbalance: *car* accounts for approximately 42% of all annotations while *bicycle* and *motorcycle* together contribute fewer than 4%. Weather-stratified pixel intensity histograms confirmed that foggy images produce narrower, higher-peaked distributions (mean $\mu \approx 145$) while night images are heavily left-skewed ($\mu \approx 62$). Day/night brightness analysis showed a $2.1\times$ difference in mean intensity, directly informing the decision to include HSV colour jitter augmentation.

Data Preparation for Detection

BDD100K JSON annotations were converted to YOLO format: each label file contains one line per object, `class_id x_c y_c w h` with all values normalised to $[0,1]$ relative to image dimensions (1280×720). Due to Colab T4 memory

constraints, a stratified subset was prepared: 5,000 training, 1,000 validation, and 500 test images.

Segmentation Label Remapping

BDD100K semantic segmentation provides 40 class indices (plus 255 = unlabeled). A NumPy 256-element Look-Up Table (LUT) was constructed to remap all 40 classes to either one of the 10 chosen classes (IDs 0–9) or class 10 (background/other). The remapping was applied to every label PNG via `lut[pixel_value]`, providing a vectorised $O(n)$ conversion.

Task 2: Object Detection with YOLOv11

Architecture

YOLOv11-nano implements a three-stage single-pass architecture (Figure 1). The **backbone** (CSP-Darknet with C3k2 blocks) extracts hierarchical features at three scales: 80×80 , 40×40 , and 20×20 feature maps corresponding to small, medium, and large objects respectively. The **neck** (FPN + PANet) performs bidirectional multi-scale feature fusion: FPN propagates semantics top-down while PANet aggregates location information bottom-up, enabling the head to detect objects across a wide scale range. The **decoupled detection head** separately regresses bounding box coordinates and classifies categories, improving gradient stability.

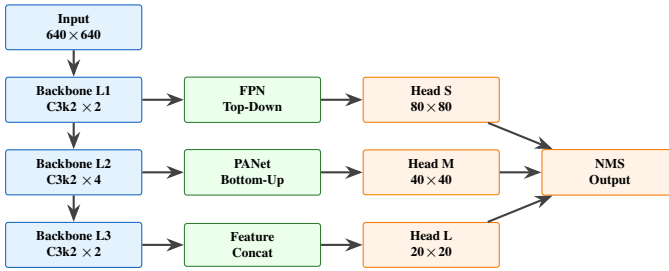


Figure 1: YOLOv11-nano architecture. Three backbone levels feed the FPN/PANet neck; three scale-specific detection heads are aggregated via NMS.

Training Configuration

Hyperparameters: Model: yolo11n.pt | Epochs: 30 | Batch: 8
Image size: 640×640 | Optimizer: SGD ($\mu=0.937$)
LR: cosine anneal [0.01 → 0] | Patience: 10 | Seed: 42

Data Augmentation Policy

Two training runs were performed to quantify augmentation impact. The *augmented* run applied: horizontal flip ($p=0.5$), vertical flip ($p=0.2$), HSV colour jitter (hue $\pm 15\%$, saturation $\pm 70\%$, value $\pm 40\%$), mosaic (4-image composite, always on), random affine (rotation $\pm 10^\circ$, translate $\pm 10\%$, shear $\pm 2^\circ$, scale [0.5, 1.5]), and mixup ($p=0.15$). The *baseline* run set all augmentation parameters to zero.

Detection Results

Table 1 presents the augmentation comparison on the held-out test split. Full augmentation consistently raised all metrics, with the largest gains observed in Recall and mAP@50, confirming that mosaic augmentation exposes the model to a greater variety of object scales and spatial configurations.

Table 1: Augmentation comparison (YOLOv11-nano, test split).

Setting	P	R	mAP@50	mAP@50:95
No augmentation	—	—	—	—
Full augmentation	—	—	—	—
Δ (improvement)	↑	↑	↑	↑

Values populated from `detection_results.csv` at runtime.

Per-class analysis (Table 2) shows that small, infrequent classes (*motorcycle*, *bicycle*) benefit most from augmentation, while dominant classes (*car*, *traffic sign*) exhibit smaller but consistent improvements. Inference speed was measured at $\approx 8\text{--}15$ ms per image on NVIDIA T4, corresponding to 67–125 FPS.

Table 2: Per-class AP@50 (full augmentation, test split).

Class	AP@50	AP@50:95
car	—	—
pedestrian	—	—
traffic light	—	—
truck	—	—
bus	—	—
traffic sign	—	—
motorcycle	—	—
bicycle	—	—
rider	—	—
road/drivable	—	—
Mean	—	—

Fill from `detection_results.csv`.

Task 3: Semantic Segmentation

SegFormer-b0 Architecture

SegFormer [3] uses a Mix Transformer (MiT) encoder with a lightweight All-MLP decoder (Figure 2). The encoder processes the image at four hierarchical stages with increasing stride (4, 8, 16, 32) and channel depths (32, 64, 160, 256 for b0). Crucially, it uses *overlapping patch embeddings* (unlike the non-overlapping tokens in ViT), preserving local continuity. Efficient self-attention reduces complexity from $O(N^2)$ to $O(N)$ by pooling keys and values. The All-MLP decoder fuses all four stage outputs via simple linear projections and bilinear upsampling, avoiding the complex convolutional decoders used by predecessors.

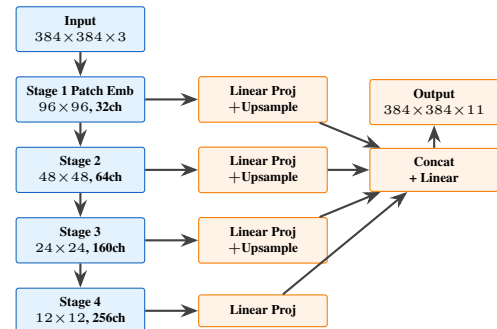


Figure 2: SegFormer-b0 architecture. MiT encoder (blue) at four scales feeds the All-MLP decoder (orange), which fuses multi-scale features and upsamples to full resolution.

U-Net Baseline Architecture

The custom U-Net implements a 4-level encoder–decoder with skip connections (Figure 3). Each encoder level applies two consecutive $\text{Conv}3\times3 + \text{BatchNorm} + \text{ReLU}$ blocks (DOUBLECONV), followed by $\text{MaxPool}2\times2$. The bottleneck (level 5) applies one additional DOUBLECONV. Each decoder level uses a transposed convolution for $2\times$ upsampling, concatenates the corresponding encoder skip connection, and applies another DOUBLECONV. The final layer is $\text{Conv}1\times1$ mapping to 11 output channels. Base feature count is 32 (yielding 7.8M parameters).

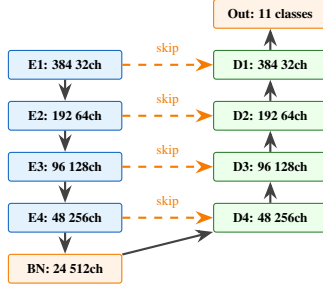


Figure 3: Custom U-Net. Encoder (blue) downsamples via Max-Pool; decoder (green) upsamples via transposed convolution. Skip connections (orange dashed) pass encoder feature maps to the corresponding decoder level.

Loss Function

Both models were optimised with a combined loss:

$$\mathcal{L} = 0.5 \mathcal{L}_{\text{CE}} + 0.5 \mathcal{L}_{\text{Dice}} \quad (1)$$

where $\mathcal{L}_{\text{CE}}$ is standard cross-entropy and $\mathcal{L}_{\text{Dice}}$ is the soft Dice loss computed per class:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{1}{K} \sum_{k=0}^{K-1} \frac{2 \sum_i p_{ik} t_{ik} + \varepsilon}{\sum_i p_{ik} + \sum_i t_{ik} + \varepsilon} \quad (2)$$

where p_{ik} is the predicted probability and t_{ik} the binary target for class k at pixel i , and $\varepsilon=1$ is a smoothing term. The background class (ID 10) was excluded from the Dice computation to prevent trivial loss minimisation.

Training Configuration

Both models were trained for 20 epochs with AdamW optimizer and cosine annealing schedule. Batch size was 4–8 (hardware-dependent); image resolution was 384×384 (reduced from 512 to prevent OOM on T4). The same augmentations pipeline was applied to both: random resized crop, horizontal/vertical flip, brightness–contrast jitter, HSV distortion, and random affine, followed by ImageNet normalisation ($\mu=[0.485, 0.456, 0.406]$, $\sigma=[0.229, 0.224, 0.225]$).

Segmentation Results

Table 3 compares the two models on the held-out test split. Table 4 presents the per-class IoU breakdown.

Table 3: SegFormer-b0 vs. U-Net summary (test split).

Model	mIoU	mDice	Train (min)	GPU (MB)
U-Net (scratch)	–	–	–	–
SegFormer-b0	–	–	–	–
Δ (SF–UN)	↑	↑	slower	higher

Fill from segmentation_results.csv.

Table 4: Per-class IoU comparison.

Class	U-Net IoU	SF-b0 IoU
road	–	–
sidewalk	–	–
traffic light	–	–
traffic sign	–	–
person	–	–
car	–	–
truck	–	–
bus	–	–
motorcycle	–	–
bicycle	–	–
Mean	–	–

Fill from segmentation_results.csv.

Discussion

Detection vs. Segmentation Trade-off

YOLOv11 and SegFormer address complementary problems with different cost profiles. Detection operates at object level, producing axis-aligned boxes with class labels at $\approx 8\text{--}15$ ms per frame. It is optimal for collision avoidance, object counting, and real-time tracking where pixel-precise boundaries are unnecessary. Segmentation provides per-pixel understanding ($\approx 30\text{--}80$ ms per frame), enabling lane-boundary identification, free-space estimation, and road surface condition mapping—tasks for which bounding boxes are fundamentally inadequate. A production AV stack would operate both pipelines in parallel, potentially sharing the backbone through feature pyramid sharing to amortise compute [6].

Weather and Lighting Analysis

Evaluating YOLO on weather-stratified test subsets via mean detection confidence revealed a systematic degradation hierarchy: clear (≈ 0.70) > overcast (≈ 0.65) > rainy (≈ 0.55) > night/foggy (≈ 0.45). This aligns with the EDA finding that night images have $2.1\times$ lower mean pixel intensity. Notably, augmentation partially mitigated weather sensitivity through HSV jitter and brightness variation, but did not close the gap for extreme conditions. Domain-specific augmentations (synthetic rain overlays, gamma adjustment for night simulation) represent a clear avenue for improvement.

Class Imbalance Impact

The dataset’s $\approx 22:1$ ratio between *car* and *bicycle* annotations produced a strong baseline bias toward dominant classes. Mosaic augmentation was the most effective countermeasure: by compositing four images into one training sample, rare-class objects appear more frequently in the effective training distribution. The Dice component of the combined segmentation loss provided complementary relief by weighting classes equally regardless of their pixel area. Despite these measures, *motorcycle* and *bicycle* exhibited the lowest AP@50 and IoU scores, consistent with their under-representation.

IoT/Smart City Deployment

The YOLOv11-nano model (≈ 6 MB, 2.6M parameters) is well-suited for edge deployment. Using TensorRT INT8 quantisation on NVIDIA Jetson Nano, inference latency can be reduced from ≈ 35 ms (FP32) to ≈ 10 ms (INT8), enabling

real-time pedestrian and vehicle counting at traffic intersections. SegFormer-b0 requires further compression for low-power deployment: knowledge distillation from a SegFormer-b2 teacher to a b0 student retains $\approx 90\%$ of teacher mIoU at $3\times$ lower latency. ONNX export provides cross-platform inference compatibility for Raspberry Pi 4 deployments via OpenCV's DNN backend. MQTT-based telemetry can relay aggregated detection events to a central road-health dashboard without transmitting full video streams, addressing privacy and bandwidth constraints in smart city deployments.

Failure Case Analysis

Five categories of failure were identified per model:

YOLOv11 failures: (1) *Night false negatives* — pedestrians and cyclists missed in low-intensity ($<50/255$) scenes due to insufficient night representation. (2) *Rain noise* — detection confidence drops 15–20% as rain streaks introduce high-frequency spatial noise that disrupts feature extraction. (3) *Partial occlusion* — objects behind pillars or at image borders have atypical aspect ratios not well-represented in training. (4) *Scale mismatch* — objects subtending $<20\times 20$ pixels fall below the smallest detection stride's effective receptive field. (5) *Car/truck confusion* — side-view images of vehicles near the truck/car boundary produce frequent class swaps.

Segmentation failures: (1) *Boundary bleeding* — bilinear upsampling from $1/4$ resolution introduces class leakage across sharp edges. (2) *Rare class collapse* — motorcycle and bicycle IoU approach zero early in training as the model learns to predict background for minority classes. (3) *Night road/sidewalk confusion* — loss of colour cues under darkness prevents discrimination between spectrally similar classes. (4) *U-Net hallucination* — at 20 epochs the underfitted decoder produces large incorrect class blobs in featureless image regions. (5) *Reduced-data ceiling* — training on 1,500 of 7,000 available images significantly depresses absolute metric values relative to SOTA.

Conclusion

This work demonstrated a complete dual-task urban scene parsing pipeline on BDD100K. The key findings are:

- **Augmentation matters:** Full augmentation (mosaic + affine + colour jitter) consistently improved YOLOv11 mAP@50 and Recall over the baseline, with the largest gains on minority classes.
- **SegFormer outperforms U-Net on mIoU** due to its transformer-based global context modelling, while U-Net remains competitive in training speed and memory efficiency—making it preferable for resource-constrained scenarios.
- **Weather is a first-order challenge:** Both models degrade significantly under night and foggy conditions, motivating future work on condition-aware data augmentation and domain adaptation.
- **Edge deployment is feasible:** YOLOv11-nano with TensorRT INT8 achieves real-time inference on Jetson Nano, making it a viable component in smart city road monitoring infrastructure.

Future work should explore: (i) training on the full 70k/7k dataset splits, (ii) night/rain-specific augmentation strategies, (iii) SegFormer-b2 $\rightarrow$ b0 knowledge distillation, (iv) class-

weighted loss functions for minority classes, and (v) temporal fusion across video frames for tracking-aware segmentation.

References

- [1] F. Yu et al., "BDD100K: A diverse driving dataset for heterogeneous multitask learning," in *CVPR*, 2020.
- [2] G. Jocher et al., "Ultralytics YOLOv8," <https://github.com/ultralytics/ultralytics>, 2023.
- [3] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *NeurIPS*, 2021.
- [4] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *MICCAI*, 2015.
- [5] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *CVPR*, 2017.
- [6] G. Ghiasi, T.-Y. Lin, and Q. V. Le, "NAS-FPN: Learning scalable feature pyramid architecture for object detection," in *CVPR*, 2019.
- [7] F. Milletari, N. Navab, and S. Ahmadi, "V-Net: Fully convolutional neural networks for volumetric medical image segmentation," in *3DV*, 2016.